

한화금융 / 김경우

기능을 넘어, 거래가 처리되는 과정까지 봅니다.

금융 시스템에서 요청이 내부 시스템을 거쳐 거래로 이어지는 과정을 이해하고,
Java와 Spring Boot로 서비스 백엔드를 직접 구현해온 개발자입니다.

SELECTED WORK

01 CRM 개발 인턴

아키텍처 설계 · RPA 리팩토링

02 강의실 예약 백엔드

동시성 제어와 정합성

03 동아리 모집 사이트

인증 흐름과 API 경계 설계

04 축제 서비스 Cardinal

지도 페이지 성능 최적화

05 교환학생 플랫폼 Dotted

기획부터 실제 운영까지

— 한화금융 / 김경우

프론트엔드에서 시작해 서비스 전체를 다뤘습니다.

프론트엔드를 중심으로 서비스의 구조와 사용자 경험을 함께 다뤘은 개발자입니다.

화면 구현에 그치지 않고, 데이터가 어떻게 흐르고 각 기능을 어디에서 처리해야 하는지 고민해왔습니다.

인턴십과 프로젝트를 거치며 Azure 기반 포털 구축, 내부 시스템 연동, 성능 개선, 운영 협업까지 경험했습니다.

WAY 01

분리

Separate Concerns

프론트엔드가 내부 시스템을 직접 알지 않도록, 인증, 저장소, 외부 연동을 각각 필요한 계층에서 처리합니다.

CRM 인턴

**Azure Function, Blob Storage 연동
구조 설계**

React / TypeScript / Azure Function
App / Blob Storage / Dataverse / Power
Automate

WAY 02

정합성

Keep Data Correct

한 번 잘 되는 것과 동시에 들어와도
맞는 것은 다르기 때문에, 문제를 먼저
재현한 뒤 애플리케이션과 DB
양쪽에서 막습니다.

강의실 예약 백엔드

비관적 락, 부분 유니크 인덱스

Django / DRF / PostgreSQL / Locust /
SonarQube

WAY 03

수명

Decide the Lifetime

기능을 만들기 전에 데이터가 어디에
얼마나 남아 있어야 하는지부터
정하고, 필요한 기간에 맞춰
관리합니다.

동아리 모집 사이트

**Redis TTL 기반 인증 흐름, 지원서 API
설계**

Java / Spring Boot / Spring Security /
JPA / Redis / MySQL

WAY 04

측정

Measure Bottlenecks

느리다는 느낌에 의존하지 않고 직접
측정하며, 여러 시도를 비교해 실제로
느려지는 지점을 찾습니다.

Cardinal

지도 페이지 INP 1130ms → 34ms 개선

React / TypeScript / Vite / TanStack
Query / Chrome DevTools / Web Vitals

WAY 05

운영

Ship & Operate

당장 동작하는 기능보다 실제로 계속
쓰일 수 있는지를 생각하고,
아이디어를 사용자에게 닿는 서비스로
발전시킵니다.

Dotted

**공모전 제안부터 국제업무부서 협업
운영까지**

React / TypeScript / Styled-
components / Supabase / GitHub
Actions

흩어진 책임과 흐름을 다시 **구조화**했습니다.

01 / CRM 개발 인턴

Azure · Dataverse · Power Platform 기반 고객사 포털 개발 및 사내 자동화 리팩토링

고객사 포털에서는 프론트엔드가 내부 시스템을 직접 알지 않도록 **API · 인증 · 저장소 계층을 분리**했고,
고객사 업무 자동화에서는 하드코딩된 흐름을 **마스터 데이터 기반 구조**로 재정리했습니다.



TOPIC 01

Pyro 고객 문의 포털

프론트에서 Dataverse를 건너낸 인증 · Blob 저장소 구조 재설계

TOPIC 02

Power Automate 리팩토링

184개 액션 → 41개, 마스터 데이터 기반 자동화 구조

TOPIC 03

Remote MCP 챗봇

개인 프로젝트로 시작해 대표 시연·사업 확장으로 이어진 AI 챗봇

Pyro Engineering

고객 포털과 CRM을 연동하는 문의 응대 서비스

BACKGROUND

고객이 문의를 남기고 처리 상태를 확인하는 웹 포털입니다. 포털 문의는 내부 CRM(Dataverse)의 Service Case로 그대로 이어져야 했습니다.

무엇을 만들었나

로그인	계정 인증 · 내 정보
문의 등록·조회	Service Case 생성 · 목록 · 상세
파일·이미지	본문 이미지 · 첨부 업로드
CRM 연동	내부 CRM에서도 확인

ROLE 단독 수행 · FE · API · Cloud · Storage

STACK React · TypeScript · Azure Function App · Blob Storage · Dataverse

PROBLEM

기능은 동작했지만, 운영 관점에서 구조적 리스크가 있었습니다.

이미지가 Dataverse에 계속 쌓인다

1 리치 텍스트 이미지가 Dataverse에 바이너리로 저장되어, 문의가 누적될수록 저장 용량을 잠식했습니다.

RISK 용량 한도 초과 시 추가 구매 · 증가 속도 예측 불가

프론트가 내부 CRM에 직접 의존한다

2 프론트엔드가 Dataverse 토큰 · URL · 응답 구조를 직접 알고 있어 내부 시스템과 강하게 묶여 있었습니다.

RISK 내부 변경이 곧 화면 코드 변경 · 토큰 노출 위험

책임이 섞여 유지보수가 어렵다

3 이미지 · 파일 · 인증 책임이 프론트와 Dataverse에 흩어져 시스템 경계가 분명하지 않았습니다.

RISK 운영 이후 구조 변경 시 마이그레이션 비용 급증

그래서 이미지 저장 책임과 내부 시스템 접근 책임을 분리하는 방향으로 재설계했습니다.

이미지 바이너리는 Blob Storage로 옮기고, Dataverse에는 HTML·메타데이터만 남겼습니다.



무엇이 달라졌나

구분	BEFORE	AFTER
이미지 저장 위치	Dataverse 첨부	Azure Blob Storage
Dataverse 역할	데이터 + 파일	데이터 + 메타데이터
용량 부담	높음 · 예측 어려움	분리로 완화
이미지 조회	Dataverse 직접	백엔드 권한 검증 후
운영 확장성	용량 증가에 취약	저장소 분리로 우수

왜 운영 전에 바꿨나

운영이 시작되면 구조는 바꾸기 어렵습니다. 이미지가 쌓이기 전에 미리 분리했습니다.

- 1

기존 이미지를 Blob Storage로 마이그레이션
- 2

본문 HTML 안의 이미지 경로를 전부 치환
- 3

이미지와 Service Case 연결 정보 재구성
- 4

CRM 화면에서 깨지는 이미지 검증
- 5

권한과 접근 정책 재설계

용량 부담 완화

바이너리를 Blob으로 분리

백엔드가 정책 제어

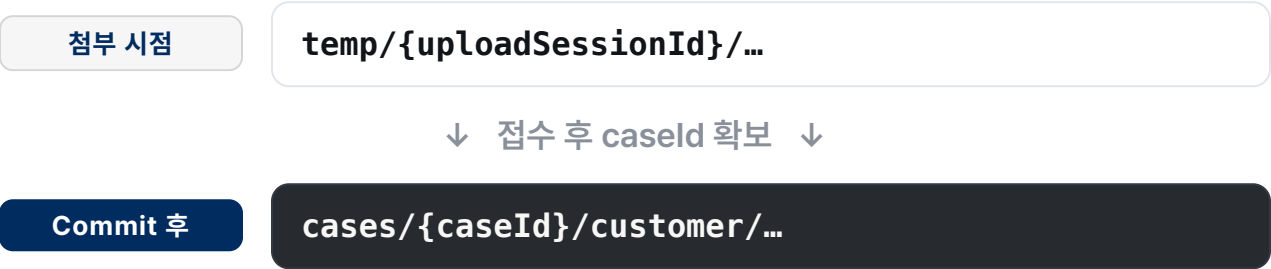
경로·메타·권한을 한 계층에서

운영 확장성 확보

저장소 분리로 안정적 확장

Temp로 먼저 올리고, Commit으로 문의에 귀속시킵니다.

작성 시점엔 최종 경로를 정할 수 없다



최종 경로는 **caseId** 기준인데, 이미지를 첨부하는 작성 중에는 아직 Service Case가 없어 caseId가 없습니다.

그래서 TEMP 단계가 필요했다

- 1

접수 전에 이탈할 수 있다

이미지만 올리고 제출을 취소하면 **Case에 연결되지 않은 이미지**가 남는다 — temp로 분리 관리.
- 2

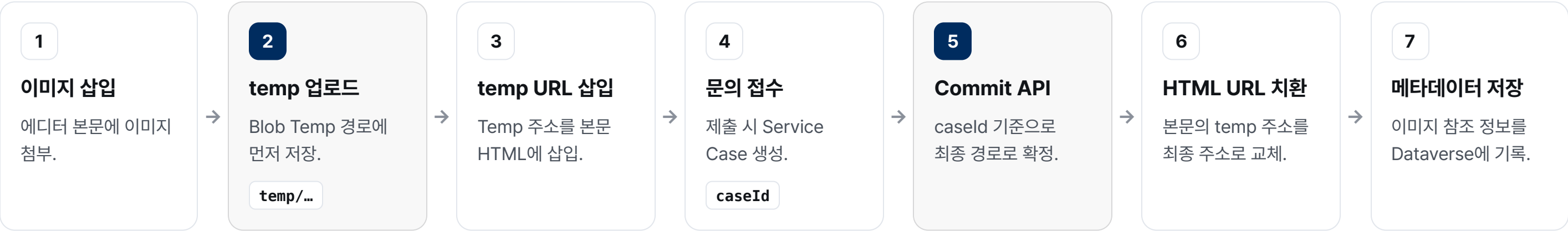
한 문의에 여러 이미지

uploadSessionId로 묶어, Commit 시 어떤 이미지들이 이 문의에 속하는지 추적한다.
- 3

최종 경로는 caseId 이후

caseId가 생긴 뒤에야 최종 경로가 정해지므로, 그전까지 **temp에 안전하게 보관**한다.

TEMP 업로드 → COMMIT · 전체 순서



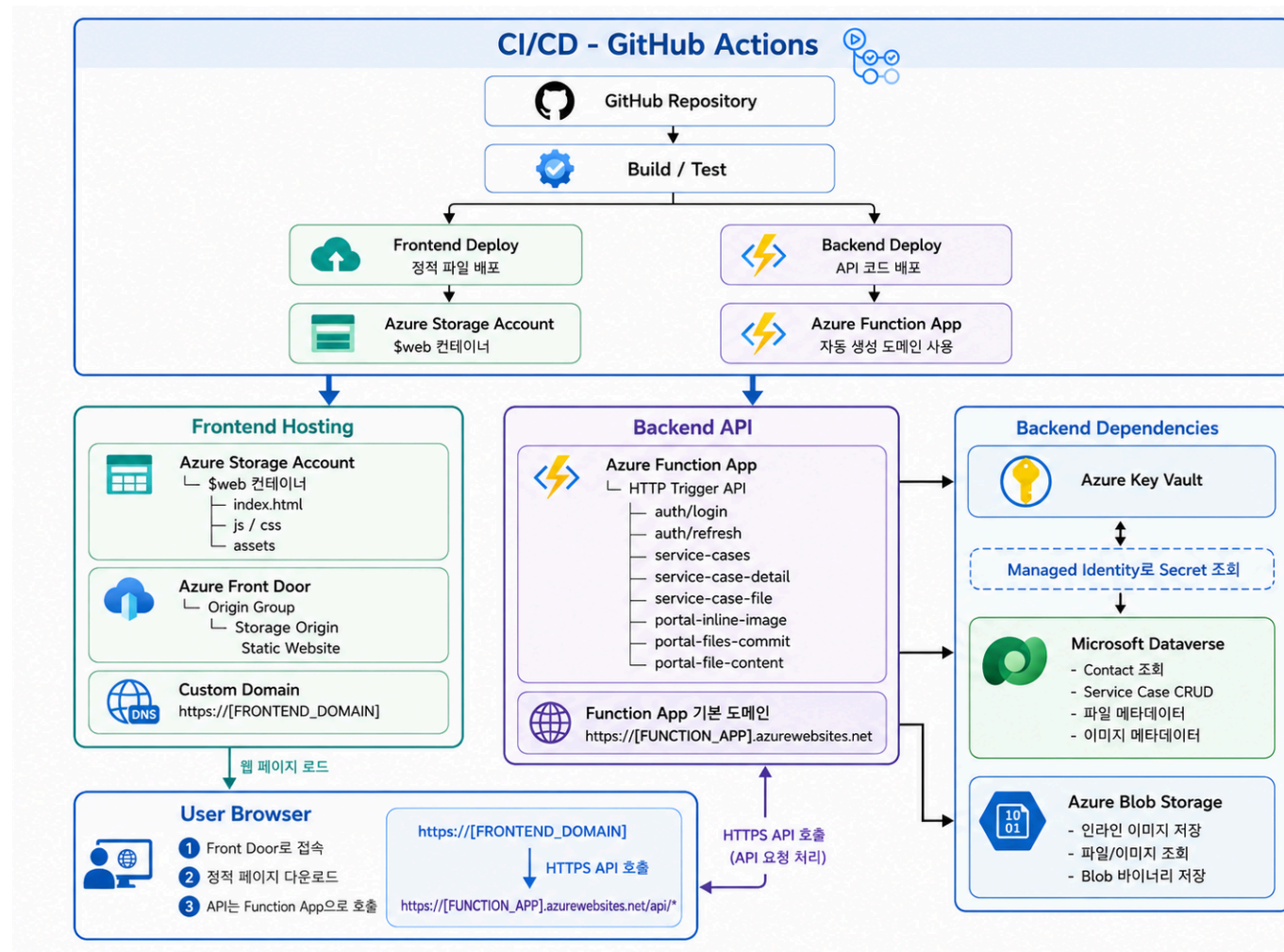
Commit은 단순 후처리가 아니라, Temp 이미지를 실제 문의에 귀속시키는 단계입니다.

작성 중 흩어져 있던 Temp 이미지가 Commit을 거쳐 하나의 Service Case로 묶입니다.

— Solution · Azure Functions 백엔드 위임 & 인증 구조

프론트는 우리 API만 호출하고,
내부 시스템 접근은 백엔드 안에서 처리합니다.

전체 시스템 아키텍처



이 구조가 보장하는 것

1 백엔드 위임

프론트는 API·JWT만, Dataverse·Blob 접근은 백엔드 안에서만.

2 두 토큰 분리

프론트는 Pyro JWT만, Dataverse Token은 백엔드 내부에서만.

3 Secret은 Key Vault에

민감 정보는 Key Vault에 두고 Managed Identity로 접근.

4 소유권 검증

JWT contactId == 리소스 owner 일 때만 케이스·파일을 반환한다.

5 CI/CD 자동 배포

GitHub Actions로 Storage·Function App에 자동 배포.

프론트는 내부 CRM 구조를 몰라도 되고,
인증·저장소 접근 정보는 백엔드 안으로 숨겨졌습니다.

— Power Automate RPA

폴더 생성 자동화를 데이터 기반 구조로 리팩토링

BACKGROUND

업무 프로젝트가 상위 프로젝트에 연결되면, SharePoint에
폴더 트리를 만들고 Dataverse에 폴더 메타데이터 레코드를
생성하는 사내 자동화입니다.

자동화가 하는 일

폴더 트리 레벨2·레벨3·하위 폴더 생성

산출물 레코드 Dataverse에 메타데이터 저장

권한 부여 계열사별 폴더 접근 권한 설정

184 → 41 actions 약 77.7% 감소

ROLE 단독 수행 · Flow 분석 · 리팩토링

STACK Power Automate · SharePoint · Dataverse

PROBLEM

동작은 했지만, 업무 규칙이 Flow 안에 하드코딩되어 바꾸기 어려운 구조였습니다.

1

폴더 하나 = 액션 하나 약 120개

레벨2·레벨3·하위 폴더가 전부 개별 액션 — 폴더 하나만 추가·변경해도 Flow를 직접 수정해야 했습니다.

2

분기·반복이 거의 없는 절차형 If 3 · Foreach 2

같은 패턴을 묶지 않고 직렬로 나열 — 액션 수가 폴더 수만큼 무한정 늘어나는 구조였습니다.

3

긴 if 식 + 문자열 payload

경로는 긴 if 중첩식, 레코드는 replace 연쇄로 조립 — 이름 하나 바뀌면 줄줄이 깨졌습니다.

LEGACY 184 ACTIONS · 구성

대부분이 단순 폴더 생성 액션

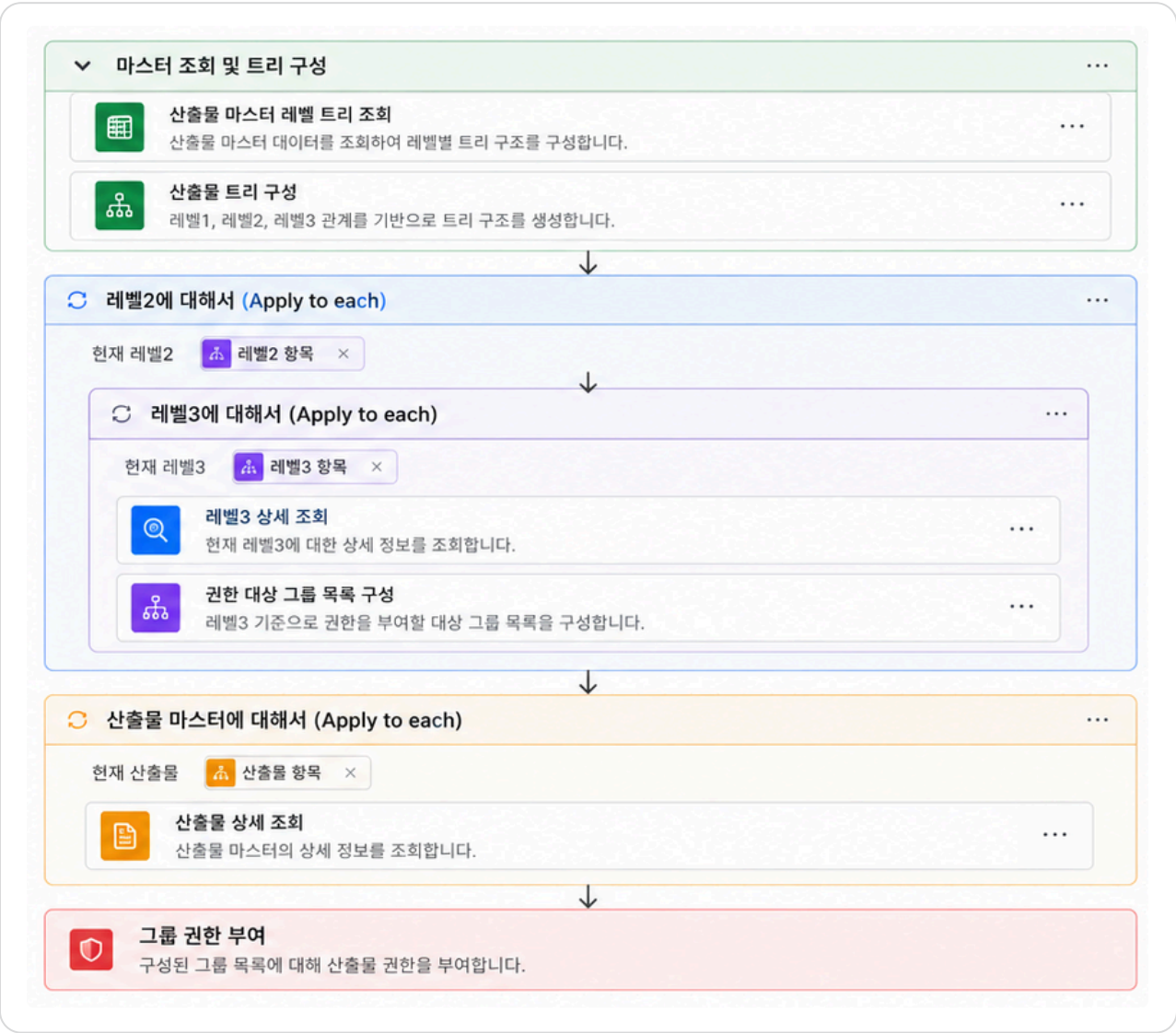


그래서 업무 규칙을 Flow에서 Dataverse 마스터 데이터로 옮기는 방향으로 리팩토링했습니다.

— Power Automate RPA 리팩토링 · 해결

업무 규칙을 **Dataverse** 마스터 데이터로 옮기고,
데이터에서 트리를 만들어 반복 생성하도록 바꿨습니다.

REFACTORED FLOW · 실제 화면



AFTER 개별 액션 나열이 **그룹화된 반복 구조**로 정돈

핵심 개선

- 1 폴더 구조를 데이터로 관리**
폴더가 바뀌어도 자동화를 고칠 필요 없이 **데이터만 수정**하면 된다.
- 2 중복 폴더 자동 정리**
같은 폴더가 여러 번 나와도 **한 번만** 만들어지도록 데이터를 정리.
- 3 데이터를 안전하게 조립·검증**
문자열 조립을 **구조화된 방식**으로 바꾸고, 쓸 수 없는 문자를 미리 걸러 오류를 방지.

41 actions	데이터 가공 액션이 새로 등장
OpenApiConnection	24
Foreach	4
Compose · 신규	5
Query · Select · 신규	5

핵심은 액션 수가 아니라 **수정 지점의 이동**
폴더가 바뀌면 Flow가 아니라 마스터 데이터만 고치면 됩니다.

— Remote MCP · 개인 프로젝트

자연어로 CRM을 물어보면 AI가 Dataverse에서 답을 찾습니다.

BACKGROUND

학습 중 알게 된 **Dataverse** 접근 **MCP**를 개인 프로젝트로 구성.
챗봇으로 시연해 사업 확장 기회로 이어졌습니다.

MCP란?

AI에 도구를 연결하는 표준 방식. AI가 CRM 테이블을 직접 탐색·
조회할 수 있다.

이게 열어줄 가능성

자연어 CRM 질의

테이블 탐색

조건 조회

관리자용 챗봇

참고 자료가 거의 없던 시점

출시 **2개월이 안 된 기능**이라, 공식 문서를 보며 직접 정리했습니다.

ROLE 개인 프로젝트 · 구성 · 챗봇 구현

STACK FastAPI · OpenAI gpt-5 · Azure AD · Dataverse MCP

구성 과정

공식 문서를 따라가며 정리한 5단계입니다.

1 Entra App 등록 Azure

Azure Entra에 앱을 등록하고 **Application(Client) ID**를 확보했습니다.

2 API 권한 부여

앱에 Dynamics CRM `mcp.tools` 권한을 추가해 **MCP 도구에 접근**하게 했습니다.

3 Power Platform에서 활성화

관리자 환경에 같은 App ID를 등록하고 **사용 허용**으로 지정했습니다.

4 Redirect URI · 포트 고정 주의한 점

`localhost:3334/callback` 으로 포트 고정. 안 하면 매번 랜덤 포트로 생성돼 인증 결과를 못 받았습니다.

5 Claude Code에 Remote MCP 연결

`claude mcp add` 로 추가하고 `/mcp` 인증 후 재시작해 **연결을 확인**했습니다.

AI에게 허용한 도구 읽기 전용 6개로 제한

`list_tables`

`describe_table`

`read_query`

`search`

`fetch`

`list_apps`

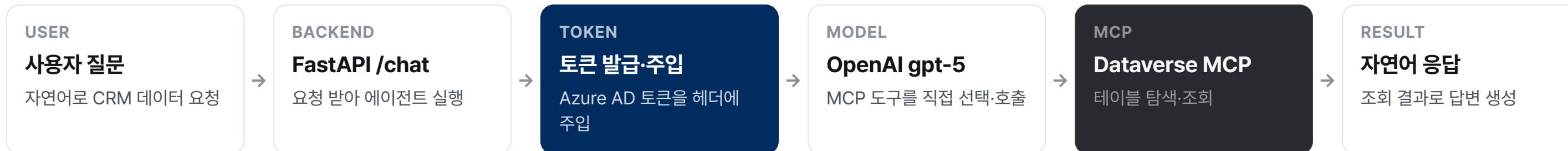
생성·수정·삭제는 빼고 **조회 도구만** 허용, `require_approval: never` 로 승인 없이 실행되게 했습니다.

토큰은 백엔드가 발급해 주입, AI가 자연어로 실제 **Dataverse** 데이터를 조회했습니다.

— Remote MCP · 구조와 임팩트

백엔드가 토큰만 넘기면, AI가 직접 Dataverse를 탐색합니다

전체 흐름 — 모델이 도구를 직접 호출한다



설계에서 신경 쓴 것

- 1 토큰은 백엔드가 발급해 주입**
Azure AD `client_credentials` 로 발급한 토큰을 헤더에 넣어, 인증 정보가 AI 쪽에 남지 않게 했습니다.
- 2 읽기 전용 도구로 제한**
조회 도구 6개만 허용하고 생성·수정·삭제는 막아 데이터를 바꿀 수 없게 했습니다.
- 3 로그인 사용자 기준 개인화**
로그인한 `user_id` 기준으로 개인 데이터는 본인 것만, 공용 데이터는 전체를 조회하도록 했습니다.
- 4 멀티턴 대화 · 호출 재시도**
사용자별 대화 히스토리를 유지하고, 호출 한도 초과 시 재시도하도록 처리했습니다.

IMPACT

개인 실험

개인 프로젝트로 MCP 구성 · 챗봇 제작

대표님 시연

자연어 CRM 조회 챗봇으로 가능성 확인



사업 확장으로 연결

개인 실험이 다른 고객사로 확장하는 사업 기회로 이어졌습니다.

프로토타입 단계라, 운영에는 사용자별 권한 · PII 마스킹 · 감사 로그가 더 필요하다고 함께 정리해 전달했습니다.

기능은 정상적으로 동작했지만, **동시 요청**에서 데이터 정합성 문제가 발생했습니다.

02 / 베틱 기반 강의실 예약 시스템

Django · DRF · PostgreSQL · 베틱 방식의 강의실 예약 백엔드

단일 요청에서는 문제가 없었지만, **같은 잔액과 같은 슬롯에 요청이 동시에** 들어오면 데이터가 달라졌습니다.
동시 요청으로 문제를 재현한 뒤, 애플리케이션과 DB에서 각각 제어했습니다.

01

앱 레벨 — 비관적 락
베틱 20건이 동시에 들어왔을 때 잔액에는 1건만 반영되던 문제를 행 잠금으로 해결

반영 1 → **20** / 20건

02

DB 레벨 — 부분 유니크 인덱스
같은 방·날짜에 여러 예약이 생성되는 문제를 DB 제약으로 차단

활성 예약 15 → **1** 건

— 베팅 기반 강의실 예약 백엔드

동시 요청에서
잔액과 예약이 어긋났습니다

BACKGROUND

마일리지를 베팅해 강의실을 배정받는 예약 서비스입니다. 잔액과 예약 상태가 정확하지 않으면 배정 결과에도 영향을 줍니다.

잔액이 변경되는 지점

베팅	잔액 차감 + 베팅 기록 + 거래내역 생성
취소	경매 진행 중이면 참여자에게 환불
예약	같은 방·날짜 슬롯을 한 팀이 점유

예약 상태 6단계

- PENDING
- BIDS_OPEN
- BIDS_CLOSED
- CONFIRMED
- REJECTED
- CANCELLED

동시 요청 시 측정 결과 (BEFORE)

차감 반영

1건

베팅 20건

활성 예약

15건

정상값 1건

ROLE 백엔드 · 동시성 제어 · 테스트 설계

STACK Django · DRF · PostgreSQL 15 · pytest

PROBLEM

단일 요청에서는 통과했지만, 동시 요청에서 값이 달라졌습니다.

- 1

잔액 차감 누락

잔액

베팅 20건은 모두 생성됐지만 잔액에는 1건만 반영됐습니다. 같은 잔액을 읽은 요청들이 각자 저장하면서 이전 차감이 덮였습니다.
- 2

한 슬롯에 예약 15건

예약

모델에 유니크 제약이 없어 같은 방·날짜에 여러 예약이 동시에 생성됐습니다.
- 3

중복 환불 가능

취소 요청이 동시에 들어오면 두 요청 모두 진행 중 상태를 읽고 환불을 실행할 수 있었습니다.
- 4

검증과 변경 시점 분리

잔액 확인과 차감 사이에 다른 요청이 들어오면 검증에 쓴 값이 이미 이전 상태가 됩니다.

수정 전에 동시 요청으로 문제를 재현했습니다.

잔액이 어긋나는 과정

Lost Update

	요청 A · 1,000 차감	요청 B · 2,000 차감	DB 잔액
t1	READ 10000	—	10000
t2	—	READ 10000	10000
t3	계산 → 9000	계산 → 8000	10000
t4	WRITE 9000	—	9000
t5	—	WRITE 8000	8000

정상값은 7,000이지만, A의 차감 1,000이 B의 저장 과정에서 덮였습니다.

Lost Update

기존 베팅 처리 reserve/views.py

BEFORE

```
# 검증이 트랜잭션 밖에서 수행됨
if request.user.mileage_balance < bet_amount:
    return Response(..., status=400)

with transaction.atomic():
    user.mileage_balance -= bet_amount
    user.save()
```

예약 전에 SELECT로 중복을 확인하면?

- A

SELECT — 중복 없음
- B

SELECT — 중복 없음
- A

INSERT
- B

INSERT — 둘 다 성공

TOCTOU — 조회와 저장 사이에 다른 요청이 들어올 수 있습니다.
transaction.atomic 만으로는 같은 행에 대한 동시 요청을 순차 처리할 수 없습니다.

— 해결 과정

앱과 DB에서 각각 동시 요청을 제어했습니다

1

비관적 락

앱 레벨

잔액 행을 잠근 뒤 검증과 차감 수행

선택

같은 잔액 행을 동시에 수정할 때 발생했습니다.
`select_for_update` 로 행을 잠근 뒤 다시 조회했습니다.

베팅 처리 `reserve/views.py`

```
with transaction.atomic():
    user = User.objects.select_for_update() \
        .get(pk=request.user.pk)

    if user.mileage_balance < bet_amount:
        return Response(..., 400)

    user.mileage_balance -= bet_amount
    user.save()
```

t1	FOR UPDATE 획득	—
t2	검증 · 차감	대기(blocked)
t3	COMMIT · 락 해제	9000 읽음

B 요청은 A의 커밋 이후 갱신된 잔액을 기준으로 처리됩니다.

취소 · 환불
예약 행을 잠근 뒤 상태를 다시 확인해 동일 예약에 환불이 두 번 실행되지 않도록 했습니다.

2

부분 유니크 인덱스

DB 레벨

저장 시점에 중복 예약 차단

선택

앱에서 미리 조회하는 방식으로는 동시 INSERT를 막을 수 없습니다. DB가 저장 시점에 검사하도록 제약을 추가했습니다.

예약 모델 `reserve/models.py`

```
UniqueConstraint(
    fields=('room', 'reservation_date'),
    condition=~models.Q(status__in=(2, 5)),
    name='unique_active_reservation...',
)
```

- 조건 없이 적용하면 취소·거절된 슬롯도 다시 예약할 수 없어 활성 상태에만 제약
- `IntegrityError` 는 **409 Conflict**로 변환해 응답합니다
- 트랜잭션은 `atomic` 범위에서 롤백합니다

상태 전이

기능별로 분산돼 있던 상태 조건을 전이 테이블로 정리하고, 종료 상태에서는 추가 전이가 발생하지 않도록 했습니다.

제약의 적용 범위
DB 제약이므로 다른 API나 직접 실행한 SQL에서도 동일한 규칙이 적용됩니다.

3

적용 후 확인

적용 후

같은 조건으로 다시 측정

결과

수정 전 사용한 것과 동일한 조건으로 다시 동시 요청을 보내 결과를 비교했습니다.

잔액 차감

동시 20건

적용 전

20건 중 1건만 반영 · 오차 19,000

적용 후

20건 모두 반영 · 오차 0

중복 예약

동시 15건

적용 전

활성 예약 15건

적용 후

활성 예약 1건 · 나머지 409

- `TransactionTestCase` — 실제 트랜잭션 동작이 필요한 테스트에 사용
- `threading.Barrier` — 여러 스레드의 요청 시작 시점을 맞춰 동시 호출

회귀 방지
동일한 동시 요청 조건을 테스트로 남겨 이후 코드 변경에서도 계속 확인할 수 있도록 했습니다.

— 결과와 회고

동시 요청에서도 데이터가 일관되게 처리됩니다

RESULT · 동일 조건 재측정

잔액 차감 반영 — 동시 20건 베팅

4 → 20 / 20건

생성된 베팅 20건이 모두 잔액에 반영됩니다.

활성 예약 — 동시 15건 요청

15 → 1 건

1건만 201, 나머지 14건은 409로 거절됩니다.
취소된 슬롯은 다시 예약할 수 있습니다.

중복 환불 차단

락 + 상태 재확인

취소 요청이 겹치면 두 번째 요청은 이미 취소된 상태로 처리됩니다.

잘못된 상태 전이 차단

전이 테이블

취소된 예약을 다시 확정하는 등 허용하지 않은 상태 변경을 막았습니다.

측정 조건

DB

PostgreSQL — 행 단위 락 사용

테스트

TransactionTestCase — 트랜잭션 경합 재현

동시성

threading.Barrier — 요청 시작 시점 동기화

WHY · 비관적 락을 선택한 이유

비관적 락을 선택한 이유

세 방식 모두 Lost Update를 막을 수 있지만, 동일한 잔액에 요청이 집중되는 상황과 검증 로직을 함께 고려해 비관적 락을 사용했습니다.

방식	특징	선택
비관적 락 select_for_update	같은 행에 경합이 발생하면 순차 처리됩니다. 별도의 충돌 재시도 로직이 필요하지 않습니다.	채택
원자적 연산 F()	단순 증감에는 가볍지만 여러 검증 조건을 함께 처리하기에는 제약이 있습니다.	대안
낙관적 락 version	충돌이 드문 환경에 적합합니다. 충돌 시 재시도 처리가 필요합니다.	대안

동일한 잔액에 요청이 집중되는 구조와 검증 조건을 고려해 비관적 락을 선택했습니다.

인증에서 중요한 건 기능보다 인증 상태를 언제까지 유지할지 정하는 것이었습니다.

03 / 동아리 부원 공식 모집 사이트

Java 17 · Spring Boot 3.4 · PostgreSQL · Redis · Docker Compose · 약 2,700명 방문

지원자를 받는 사이트는 모집 기간에만 운영되고, 특정 시간대에 방문이 몰립니다.
외부 인증 API에 의존하던 본인 확인을 직접 관리하는 인증 방식으로 바꾸고, 인증번호의 유효시간을 Redis에서 관리했습니다.

01

인증 과정을 직접 관리

외부 인증 API에 맡기던 본인 확인을 메일 발송 + Redis 검증 방식으로 바꾸고, 인증번호 만료와 재발급 기준을 직접 설정했습니다

외부 API 의존 → 자체 인증 / 인증 정책 직접 관리

02

인증번호는 TTL로 자동 만료

인증번호는 일정 시간이 지나면 사용할 수 없어야 하는 데이터이므로 별도 삭제 로직 대신 Redis TTL을 적용해 자동으로 만료되도록 했습니다

TTL · 30분 / 재발급 60초 · 최대 3회

— 동아리 부원 공식 모집 사이트

인증 정책을 직접 제어할 수 없었습니다

BACKGROUND

지원자가 본인 확인 후 지원서를 제출하고 결과를 조회하는 사이트입니다. 모집 기간에만 운영되며 짧은 기간에 접속이 몰립니다.

사용자 요청 흐름

본인 확인 인증번호 발송 → 입력 → 검증

지원서 생성 / 조회 / 수정

결과 조회 서류 / 최종 결과 확인

지원 상태

- 지원함
- 서류합격
- 서류탈락
- 최종합격
- 최종탈락

개발 시점의 조건

개발 기간 2주 백엔드 4인

지원 규모 예측 불가 모집 공고 후 결정

ROLE 백엔드 · 인증 흐름 · 지원서 API

STACK Java 17 · Spring Boot 3.4 · JPA · PostgreSQL · Redis

PROBLEM

인증은 됐지만, 필요한 정책을 직접 적용하기 어려웠습니다.

- 1

만료 시간을 정할 수 없음 **인증**

발급과 검증이 외부 API에서 처리돼, 인증번호를 몇 분 유지할지 서비스에서 정할 수 없었습니다.
- 2

재발송을 제한할 수 없음 **인증**

같은 이메일로 계속 요청할 수 있었고, 요청한 만큼 메일도 반복 발송됐습니다.
- 3

비밀번호가 평문으로 저장

재설정 시 입력값을 그대로 저장했습니다. 지원서에는 학번·연락처도 함께 저장됩니다.
- 4

실패 원인을 구분하기 어려움

만료·불일치·중복의 처리 방식이 일정하지 않아 프론트에서 원인을 구분하기 어려웠습니다.

정리한 질문

Q1

인증번호를 어디에 저장해야 시간이 지나면 자동으로 삭제되는가

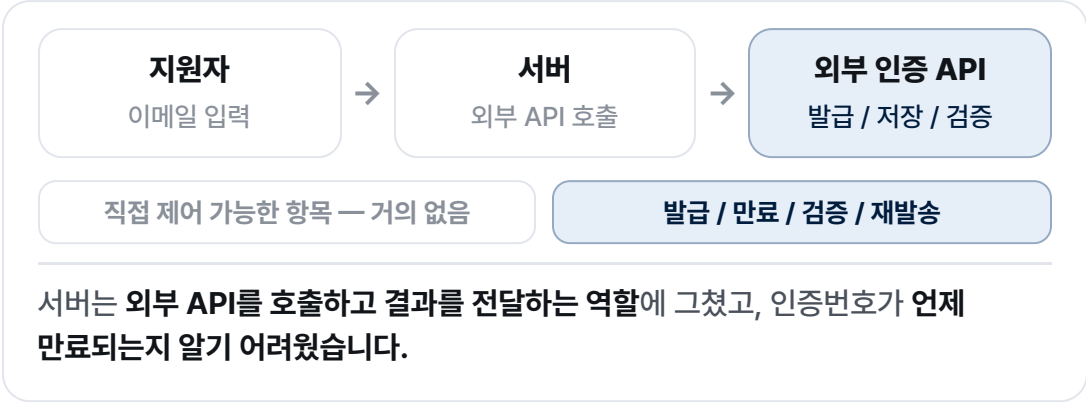
Q2

만료, 불일치, 미발급을 어떻게 구분해 전달할 것인가

인증번호는 일정 시간이 지나면 반드시 사라져야 하는 데이터였습니다.

기존 인증 구조

외부 API 위임



기존 인증 처리 AuthService.java

BEFORE

```
// 발급과 검증이 외부 API 호출로 끝남
Map<String, Object> result =
    UnivCert.certifyCode(key, email, univName, code);

if ((boolean) result.get("success")) { ... }

// 재설정 시 입력값을 그대로 저장
app.updatePassword(requestDto.getPassword()); // 평문
```

지원이 몰릴 때 반복되던 상황

1

인증번호 요청 → 메일 도착 지연

발송

2

다시 요청 → 이전 인증번호도 그대로 유효

발송

재발송 기준이 없어 요청할 때마다 새로운 메일이 발송됐습니다.

이 구조에서 확인하기 어려웠던 것

•

인증번호가 언제 만료되는지 — 서비스에서 알 수 없었습니다.

•

한 사용자가 몇 번 요청했는지 — 별도로 관리하지 않았습니다.

— 해결 과정

인증번호의 만료, 횟수, 검증을 직접 관리했습니다

외부 API에 맡기던 인증을 서비스에서 처리하도록 바꾸고,
만료 시간과 재발송 기준을 직접 설정했습니다.

AFTER — 인증 흐름

/auth

STEP 01
발송 요청

이메일과 요청 유형을 확인합니다.

POST /send-code/{signup,reset}

STEP 02
횟수 제한

이메일별 요청을 기록해 60초 동안 3회를 넘으면 차단합니다.

email_limit_{email} · TTL 60s

STEP 03
저장

SecureRandom으로 6자리 인증번호를 생성해 Redis에 30분간 저장합니다.

SET {email} = code · TTL 30m

STEP 04
검증

값이 없으면 만료, 다르면 불일치로 구분합니다.

GET {email} → null? / equals?

STEP 05
사용

인증 후 지원서 생성 또는 비밀번호 재설정으로 이어집니다.

/application · /reset-password

1 Redis TTL로 만료 처리

발급 시간을 저장해 매번 계산하는 대신, 30분 뒤 값이 자동으로 삭제되도록 했습니다.
MailService.java

```
// 코드 생성 + 수명을 함께 지정
String authCode = createCode(); // SecureRandom 6자리
redisUtil.setDataExpire(email, authCode, 60 * 30L);

// 검증 - 값이 없으면 곧 만료
String found = redisUtil.getData(email);
if (found == null) throw new VerificationFailedException(...);
```

인증번호를 저장할 때 만료 시간까지 함께 지정해, 시간이 지나면 Redis에서 값이 자동으로 사라집니다. 검증 시에는 별도의 시간 계산 없이 값이 남아 있지만 확인했습니다.

결과
별도 삭제 로직 없이 인증번호 만료를 처리했습니다.

2 실패 원인을 구분

만료와 불일치는 사용자가 해야 할 행동이 다르기 때문에 저장된 값의 상태에 따라 결과를 나눴습니다.

저장된 값의 상태 → 결과		
없음	TTL로 삭제됨	만료
값 다름	저장된 값과 입력이 다름	불일치
값 같음	유효 시간 안에 같은 값	통과

재발송
기존 번호를 삭제한 뒤 새 번호를 저장해, 유효한 인증번호가 하나만 남도록 했습니다.

결과
프론트에서 만료, 불일치, 횟수 초과를 구분해 안내할 수 있게 됐습니다.

3 기능별 책임 분리

MailService
인증번호 발급 / 저장 / 검증 / 재발송 제한

PasswordService
BCrypt 해싱 / 비밀번호 비교

ApplicationService
지원서 생성 / 조회 / 수정

ResultService
지원 상태에 따른 결과 조회

비밀번호도 평문 저장에서 BCrypt 해싱 방식으로 변경하고, 기존과 같은 값은 사용할 수 없도록 했습니다.

결과
인증 관련 변경 사항을 MailService 중심으로 관리할 수 있게 됐습니다.

— 결과

인증 흐름을 API별로 나눠 구현했습니다

구현한 API

POST	/auth/send-code/{signup,reset}	가입 여부 확인 후 인증번호 발송 / 60초 3회 제한
POST	/auth/verify-code	Redis 값으로 만료 / 불일치 / 통과 구분
POST	/auth/reset-password	BCrypt 저장 / 기존 비밀번호와 같으면 거절
POST	/application · /application/view	지원서 생성 / 조회 / 조회 전 비밀번호 확인
PUT	/application	이메일, 비밀번호, 지원 상태는 기존 값 유지
POST	/result/{document,final}	지원 상태에 따라 조회 가능한 결과만 반환

예외 처리 표준화		GlobalExceptionHandler
400	VerificationFailed / DuplicateEmail	
401	InvalidPassword	
403	InvalidUserStatus	
429	EmailLimitExceeded	
모든 API 결과를 success / code / message / data 형식으로 통일했습니다.		

운영 결과		
총 방문자	2,700명	모집 기간 집중
인증번호 유효시간	30분	Redis TTL 자동 만료

WHY / REDIS를 선택한 이유

인증번호는 일정 시간이 지나면 없어져야 하는 데이터입니다

만료 시간을 매번 계산하거나 별도 삭제 작업을 두는 대신, TTL로 자동 삭제되는 Redis를 선택했습니다.

방식	특징	
Redis TTL setDataExpire	시간이 지나면 자동 삭제되어 별도 만료 계산이나 삭제 작업이 필요 없습니다.	채택
DB 테이블 created_at 비교	관리하기 쉽지만 만료된 값이 남아 별도 정리가 필요합니다.	대안
외부 인증 API UnivCert	적용은 빠르지만 만료와 재발송 기준을 직접 정하기 어렵습니다.	이전

더 다뤄 볼 부분

- 재발송 제한이 이메일 기준이라 다른 주소로 우회할 수 있습니다.
- 지원서 수정 시 실제 인증 완료 여부를 서버가 다시 확인하는 절차가 필요합니다.
- 동일 지원서에 수정 요청이 동시에 들어오는 상황은 별도로 검증하지 못했습니다.
- 인증 완료 여부를 짧은 유효시간의 토큰으로 남기면 이후 요청에서도 확인할 수 있습니다.

회고

처음에는 인증을 메일을 보내고 값을 확인하는 기능으로 생각했습니다. 직접 구현하면서 인증번호를 어디에 저장하고, 얼마나 유지하고, 언제 없앨지까지 함께 설계해야 한다는 것을 배웠습니다.

체감이 아니라 **측정값으로** 병목을 확인했습니다.

04 / 학교 축제 서비스 Cardinal

React · TypeScript · Mapbox 기반 축제 지도 페이지 성능 개선 · 프론트엔드

사용자가 자주 이용하는 지도 페이지에서 **리스트를 여닫을 때** 심한 **프레임 드롭**이 발생했습니다.

가설을 세우고 측정한 뒤, 효과가 부족하면 다시 원인을 분석하며 **병목 지점을** 좁혀갔습니다.

— 학교 축제 지도 페이지

마커·리스트를 열 때 끊기는 지도 화면

BACKGROUND

동아리에서 개발한 학교 축제 웹 서비스로, Mapbox 지도 위에 9개 카테고리 마커와 부스 리스트를 구현했습니다.

지도 페이지 주요 기능

카테고리	선택 시 해당 마커만 지도에 표시
마커 클릭	주점·푸드트럭 등 부스 리스트를 드로어로 표시
드로어	높이를 20%→80%로 조절하는 인터랙션

INTERACTION (INP)

약 97% 단축

1130 → 343 → 34ms

ROLE 프론트엔드 · 지도 페이지 · 성능 측정·개선

STACK React · TypeScript · Mapbox · framer-motion

PROBLEM

기능은 동작했지만, 리스트를 여닫을 때 화면이 끊겼습니다.

- 1

드로어 애니메이션 프레임 드롭

리스트를 열고 닫는 과정에서 심한 프레임 드롭이 발생했습니다.
- 2

사용 빈도가 높은 화면

지도와 부스 목록은 사용자가 반복해서 조작하는 화면이라 지연이 바로 체감됐습니다.
- 3

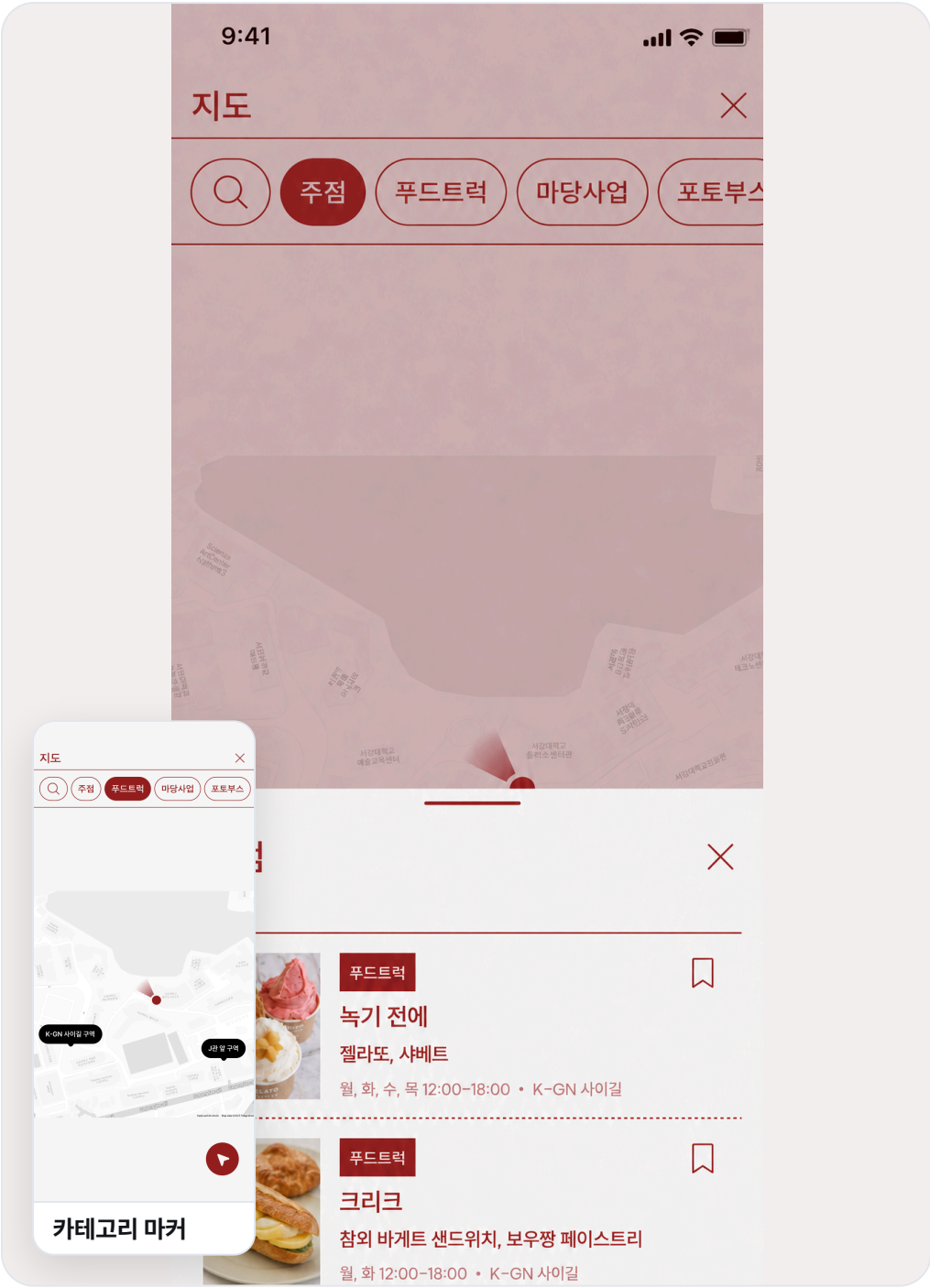
일반 지표만으로 원인 확인 어려움

Lighthouse의 LCP·TBT만으로는 해당 인터랙션 지연을 설명하기 어려웠습니다.

Performance 탭과 Clarity를 추가로 확인하며 병목을 좁혔습니다.

문제가 발생한 화면

마커 → 부스 리스트 드로어



마커 클릭 → 부스 리스트 드로어 · 여닫을 때 프레임 드롭 발생

— 병목 추적

병목 원인을 단계별로 확인했습니다

1 Lazy Load

개선 제한

이미지 요청 시점 늦추기

가설

한 번에 많은 이미지를 불러오는 것이 원인이라고 보고
lazy loading을 먼저 적용했습니다.

리스트의 썸네일

```
<img
  src={src}
  loading="lazy"
  decoding="async"
  fetchPriority="low"
/> // 펼치면 전체 항목은 DOM에 마운트
```

- 기대 — 화면 밖 이미지 요청을 미뤄 초기 처리량 감소
- 실측 — 체감은 일부 개선됐지만 지표 변화는 제한적

확인

lazy 는 요청 시점을 조절하지만, 드로어를 열 때 **항목 전체가 DOM에 생성**되는 구조는 그대로였습니다.

2 IntersectionObserver

부분 개선

화면 근처 이미지만 렌더링

시도

IntersectionObserver로 화면 근처에 들어온 항목만
 를 생성하도록 변경했습니다.

LazyImage.tsx

```
const io = new IntersectionObserver((e) => {
  if (e[0].isIntersecting) {
    setInView(true);
    io.unobserve(e[0].target);
  }
}, { root, rootMargin, threshold });

{inView && !error && <img src={src} ... />}
```

- rootMargin 으로 화면 진입 전에 미리 로드
- 감지 후 unobserve 로 불필요한 재관찰 방지
- 재사용 컴포넌트에 memo + skeleton 적용

INP 1130 → 343MS

열 때는 개선됐지만 닫을 때는 여전히 프레임 드롭이 발생해, 이미지 개수 외의 **처리 비용**을 확인했습니다.

3 원본 이미지 해상도

주요 병목

Network 탭에서 이미지 크기 확인

발견

이미지 **응답 시간이 길어** 원본 파일을 확인한 결과, 실제 표시 크기에 비해 **해상도가 지나치게 컸습니다.**

6000×4000

약 2,400만 픽셀

- 표시 크기 **600×400** 대비 픽셀 수 약 **100배**
- 화면에서 작게 표시되도 **고해상도 원본**을 처리한 뒤 축소

개선안

측정 결과를 정리해 업로드 시 **AWS Lambda로 이미지를 자동 리사이즈**하는 방식을 백엔드에 제안했습니다.

병목 확인

고해상도 이미지 처리 비용이 드로어 애니메이션과 겹치면서 응답 지연과 프레임 드롭이 발생했습니다.

— 개선과 결과

프론트와 이미지 처리 구조를 함께 개선했습니다

HOW · 처리 대상 수와 이미지 크기를 각각 줄였습니다

FRONTEND · 담당

1130 → 343 ms

렌더링 대상 수 감소

IntersectionObserver로 화면 근처 항목만 `<img>` 를 생성해 한 번에 처리하는 이미지 수를 줄였습니다.

프론트 개선 이후에도 지연이 남았습니다.

화면에 보이는 이미지 자체가 6000×4000 원본이어서 이미지 처리 비용이 계속 발생했습니다.

BACKEND · 제안

343 → 34 ms

이미지 리사이즈 제안

측정 결과와 원본 이미지 크기를 공유하고, 업로드 시 **AWS Lambda**에서 표시 크기에 맞게 자동 리사이즈하도록 제안했습니다.

프론트에서는 처리할 이미지 수를 줄이고, 백엔드에서는 이미지 한 장의 처리 비용을 줄이는 방식으로 개선했습니다.

RESULT

INP — 마커 클릭 후 응답

1130 → 34 ms

처리 대상 수와 이미지 크기를 함께 줄여 약 97% 단축

• 프레임 드롭 개선

반복 드롭 → 정상 동작

긴 리스트를 닫을 때도 이전보다 끊김 없이 동작했습니다.

• 이미지 처리량 감소

2,400만 → 24만 px

표시 크기에 맞게 리사이즈해 이미지당 픽셀 수를 약 1/100로 축소했습니다.

• 네트워크 전송량 감소

원본 이미지 전송량 ↓

화면에 필요한 이미지만 표시 크기에 맞는 해상도로 받아 전송량도 줄였습니다.

• 사용자 행동 확인

MS Clarity

세션 녹화에서 개선 전후를 비교하고 반복 클릭 감소를 확인했습니다.

측정값을 기준으로 원인을 좁히고, 프론트와 이미지 처리 구조를 함께 개선했습니다.

기획에서 출발해, 수상과 협업을 거쳐 실제 운영까지 이어졌습니다.

05 / 국제학생 정보 플랫폼 Dotted

React · TypeScript 기반 국제학생 정보 교류 플랫폼 · 프론트엔드 · 기획·외부 협업 참여

교내에서 발견한 국제학생 정보 접근 문제를 서비스로 구현하고,
공모전 대상과 국제처 협업을 거쳐 약 100명이 쓰는 서비스로 운영한 가장 긴 프로젝트입니다.

성과

융합기술 경진대회 대상

총장상

국제처 협업

약 100명 사용자

약 7개월 운영

— 국제학생 정보 플랫폼

필요한 정보를 한곳에서 찾기 어려웠다

BACKGROUND

외국인 국제학생을 위한 **정보 교류 플랫폼**입니다. 학교 생활에 필요한 정보를 모으고, 학생끼리 직접 묻고 공유할 수 있도록 만들었습니다.

DOTTED가 하는 일

Tips	캠퍼스 지도·식당·병원·FAQ·동아리 정보 제공
Community	카테고리·검색 게시판으로 직접 묻고 공유
Market	국제학생 간 중고 거래

ROLE 프론트엔드 · 로그인·메인·게시판 CRUD · 기획·외부 협업

STACK React · TypeScript · React Query

PROBLEM

정보가 없는 게 아니라 흩어져 있어, 새로 온 학생이 찾기 어려웠습니다.

- 1

공식 정보가 흩어져 있다
수강신청·기숙사·행정 안내가 여러 부서 페이지·한국어 문서에 나뉘어, 영어로 한 번에 보기 어려웠습니다.
- 2

생활 정보는 사람 사이에만
식당·병원 같은 실제 팁은 선배·학생 사이에만 있어, 새로 온 학생은 매 학기 같은 질문을 반복했습니다.
- 3

신뢰·최신성 판단이 안 됨
검색해도 작년 정보인지 지금도 유효한지 구분이 어려워, 잘못된 정보로 헤매기 쉬웠습니다.

SO WE BUILT — DOTTED

탐색·공유·거래를 한 서비스로

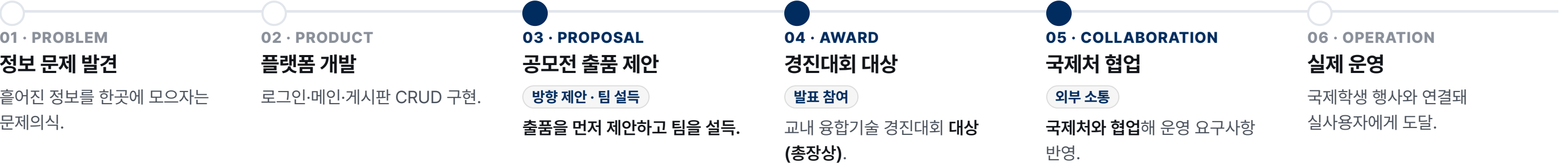
Tips 흩어진 정보를 카테고리 로 정리해 한곳에서	Community 반복 질문을 학생끼리 직접 해소	Market 단기 체류에 필요한 물품 을 거래
--	--	--

| 정보의 부재가 아니라 분산이 문제 — 한 서비스로 모은 뒤, 실제 운영까지 이어갔습니다.

— 기회로 연결하다

개발 결과물에서 멈추지 않고, 공모전과 협업으로 밀고 갔습니다.

PROJECT → OPERATION



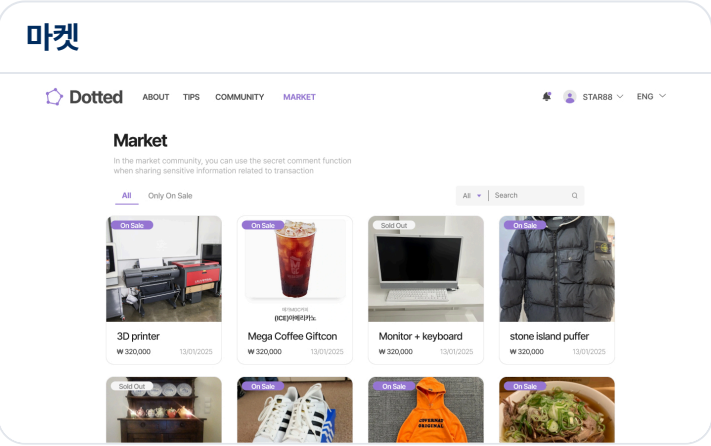
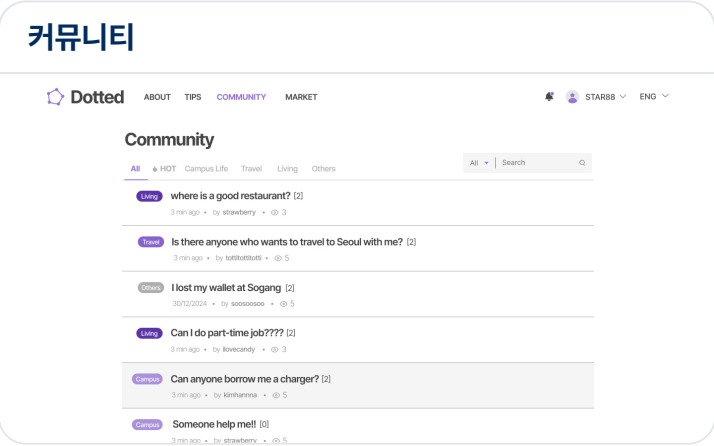
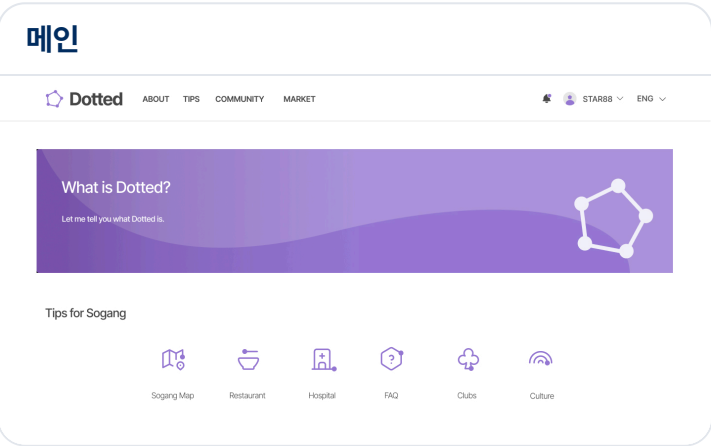
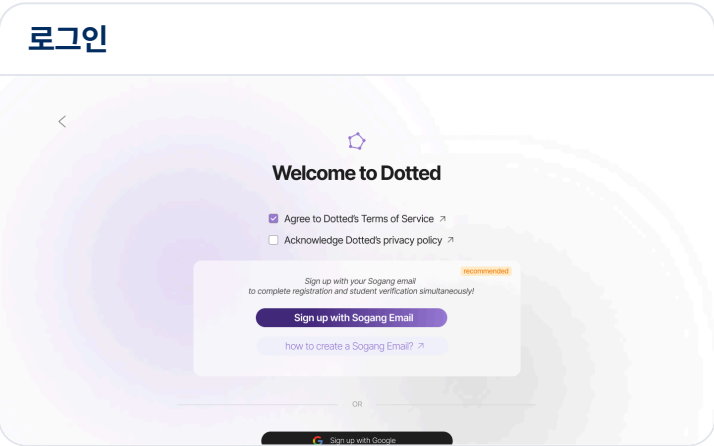
교내 융합기술 경진대회

대상

인문사회·예체능·융합 부문 · 총장상

부문
전체 융합 부문 중 최고상

이후
국제처 공식 협업으로 연결



OPERATION · IMPACT

~100명

실제 사용자

~7개월

실서비스 운영

~9개월

기획→운영, 가장 긴 프로젝트

기능 구현에서 멈추지 않고 제안·설득·외부 협업으로 실제 운영까지 넓혔습니다.

운영을 겪으며 문서화·커뮤니케이션도 운영의 일부임을 배웠습니다.

THANK YOU

읽어주셔서 감사합니다

궁금한 점이나 함께 이야기하고 싶은 부분이 있다면
언제든 편하게 연락 주세요.

EMAIL

shu07002@gmail.com

PHONE

+82 10-3997-5917

GITHUB

github.com/shu07002

BLOG

shu07002.tistory.com